

Binary Heap vs Balanced Binary Search Tree

Name: Deepak R Reg. No.: 192511008

Question

Compare a binary heap (priority queue) versus a balanced binary search tree (e.g., AVL or Red-Black) for implementing a scheduler that requires both min extraction and arbitrary key updates. Analyze time complexity for insert, extract-min, and decrease-key operations. Which data structure is more memory-efficient?

1. Binary Heap (Priority Queue)

A binary min-heap stores the smallest key at the root. It is usually implemented using an array, so the elements occupy contiguous memory. This makes it efficient for a scheduler where the next task is obtained by extracting the minimum-priority task.

Insert: $O(\log n)$ because the new element may move upward to restore the heap property.

Extract-min: $O(\log n)$ because the root is removed and the last element is moved down.

Decrease-key: $O(\log n)$ if the position of the item is known; the item can then move upward.

2. Balanced Binary Search Tree

An AVL tree or Red-Black tree maintains a balanced binary-search-tree ordering. This means keys can be searched, inserted, deleted, and updated while keeping the tree height $O(\log n)$. A minimum element can be found by following the leftmost path.

Insert: $O(\log n)$, including the required balancing operations.

Extract-min: $O(\log n)$, because the minimum node is removed and the tree may need rebalancing.

Decrease-key: $O(\log n)$ when the affected node can be located; if decreasing a key violates BST ordering, the node may need to be removed and reinserted, which is still $O(\log n)$.

3. Time Complexity Comparison

Operation	Binary Min-Heap	Balanced BST (AVL / Red-Black)	Observation
Insert	$O(\log n)$	$O(\log n)$	Both are logarithmic.
Extract-min	$O(\log n)$	$O(\log n)$	Heap has a simpler and direct minimum extraction.
Decrease-key	$O(\log n)^*$	$O(\log n)$	Heap needs the item's position/handle.
Find/search arbitrary key	$O(n)$	$O(\log n)$	BST is better for arbitrary-key lookup.

*For a binary heap, decrease-key is $O(\log n)$ when the scheduler maintains a reference/index (handle) to the task. Without knowing where the task is stored, locating it may take $O(n)$.

4. Memory Efficiency

The binary heap is generally more memory-efficient. It can be stored directly in an array with no explicit left-child, right-child, parent, or balance/color pointers. A balanced BST normally stores each node as a separate object and requires child pointers and, depending on the implementation, additional metadata such as height or color. Therefore, for a scheduler primarily concerned with priority ordering, the heap has lower memory overhead.

5. Best Choice for the Scheduler

Binary heap is the better choice when the scheduler mainly needs to insert tasks, repeatedly extract the minimum-priority task, and perform decrease-key updates. It provides $O(\log n)$ for all three required operations (with a maintained task handle for decrease-key), has a simple implementation, good cache locality, and lower memory overhead.

A balanced BST becomes preferable when the scheduler also needs efficient searching for arbitrary keys, ordered traversal, range queries, predecessor/successor operations, or frequent arbitrary-key deletion. Thus, the choice depends on the complete operation set, but for the stated scheduler requirements, the binary heap is more suitable and memory-efficient.

Conclusion

Both structures provide $O(\log n)$ insertion and minimum extraction, and both can support $O(\log n)$ key-decrease updates under suitable assumptions. However, a binary heap is more compact and has lower memory overhead. Therefore, for a priority-based scheduler requiring min extraction and decrease-key operations, a binary min-heap is the recommended data structure.